

Relatório BixoQuest: Da Matrícula à Formatura.

Pedro Haywanon Santos Araujo¹

¹Engenharia de Computação
Universidade Estadual de Feira de Santana (UEFS)
Feira de Santana, BA – Brasil

peuhsa@gmail.com

Resumo. *Este projeto apresenta o desenvolvimento do jogo BixoQuest: Da Matrícula à Formatura, uma simulação universitária em Java construída com princípios de Orientação a Objetos e padrão MVC. Foram aplicados conceitos de herança, polimorfismo e classes abstratas na modelagem dos componentes do jogo, como personagens, locais, eventos e NPCs. A primeira fase contempla a implementação do model, testes de unidade com JUnit e o diagrama de classes UML. O jogo simula a trajetória de um estudante de Engenharia de Computação da UEFS, desde a matrícula até a formatura, com sistema de turnos baseado em dias e semestres, eventos aleatórios e obrigatórios, gerenciamento de atributos do personagem e interação com o ambiente universitário.*

1. Introdução

O texto a seguir tem como objetivo demonstrar a fase inicial de construção do jogo BixoQuest: Da matrícula à formatura, relatando decisões de projetos implementadas, problemas encontrados, testes de unidade desenvolvidos, diagrama de classes e conceitos utilizados relacionados à linguagem de programação Java.

O problema central consiste em modelar, utilizando os princípios da Programação Orientada a Objetos e o padrão MVC, um jogo de simulação universitária funcional. O jogador é um estudante calouro de Engenharia de Computação da UEFS, que precisa simular a vida universitária, desde o ingresso até a formatura. Ao longo do jogo, ele enfrenta diversos desafios, assim como exige a vida universitária, como provas, cansaço, falta de dinheiro, eventos inesperados, enquanto explora o campus, interage com NPCs e gerencia seus atributos.

O jogo proporciona uma experiência divertida de simulação para o estudante universitário, com decisões reais e que impactarão no seu dia-a-dia. A solução proposta organiza os componentes do jogo em classes coesas e hierarquias bem definidas, separando responsabilidades entre model, view e controller, de forma a facilitar a manutenção e a evolução do sistema nas fases seguintes do projeto.

2. Fundamentação Teórica

Nas subseções abaixo, será descrito a fundamentação teórica utilizada para solução e desenvolvimento do algoritmo de simulação universitária, com o objetivo de contextualizar e entender o que foi necessário para o sucesso da fase inicial. A subseção Diagrama de Classes vai clarear conceitos sobre análise e projetos de sistemas, assim como a UML, MVC. A subseção Linguagem de Programação Java abordará conceitos fundamentais

como Programação Orientada a Objetos, UML e Diagrama de Classes. Além disso, terá uma subseção falando sobre o que são testes de unidade e como utilizamos para melhoria do sistema.

2.1. Linguagem de Programação Java

A linguagem de programação Java é altamente aceita e utilizada, especialmente em servidores de backend para programação, por ser independente de plataformas, possuir uma biblioteca rica e ser orientada a objetos.

2.1.1. Programação Orientada a Objetos

A Programação Orientada a Objetos (POO) é um paradigma de programação baseado na organização do sistema classes e objetos. As classes funcionam como moldes do produto, definindo atributos (características) e métodos (comportamentos), enquanto os objetos são instâncias dessas classes. Isso favorece a manutenção e entendimento do projeto, pois compartimentaliza as funções, favorecendo a alta coesão e baixo acoplamento.

Os principais pilares do POO utilizados nesse programa foram: **herança**, que permite que uma classe filha herde atributos e métodos de uma classe pai, evitando repetição de código; **polimorfismo**, que permite que classes diferentes respondam ao mesmo método de formas distintas; **encapsulamento**, que protege os atributos de acesso direto externo por meio de getters e setters com regras de validação, como os limites de energia e motivação do jogador; e **abstração**, que permite definir classes e métodos abstratos como contratos.

2.1.2. UML

“A UML, Linguagem Unificada de Modelagem, é uma linguagem gráfica para visualização, especificação, construção e documentação de artefatos de sistemas complexos de software.” [BOOCH et al. 2005]. Na especificidade do problema atual, a UML que foi utilizada foi o diagrama de classes, visto que a linguagem de programação Java é completamente orientada à objetos. Dessa forma, a documentação, desenvolvimento e organização do problema ficou mais fácil.

2.1.3. Diagrama De Classes

“Um diagrama de classe é um diagrama que mostra um conjunto de classes, interfaces e colaborações e seus relacionamentos. Graficamente, um diagrama de classes é uma coleção de vértices e arcos.” [BOOCH et al. 2005]. Geralmente, tem classes, interfaces e setas mostrando suas relações de herança, associação, agregação e dependência. São uma representação estática da estrutura do seu sistema. Apesar de deixar a modelagem do algoritmo mais fácil, pois há um planejamento antecedente a codificação, um diagrama de classes não é parte essencial do código, nem da estruturação de um sistema.

O diagrama de classes completo pode ser visualizado no anexo A (Figura 6).

2.2. Padrão MVC

Um padrão de projeto é uma solução já testada que resolve um problema específico em projetos diferentes. Todo projetista deseja manter um padrão de alta coesão e baixo acoplamento, para melhor manutenção, visualização, compartimentalização e entendimento dentro do seu código.

O padrão MVC (*Model-View-Controller*) é apresentado em 1979 por Trygve Reenskaug, como uma forma de modelagem dividida em três camadas. "O *view* gerencia a saída gráfica e/ou textual na parte da tela bitmap que é destinada à sua aplicação. O *controller* interpreta as entradas de mouse e teclado do usuário, comandando o *model* e/ou a *view* para que mudem conforme necessário. Por fim, o *model* gerencia o comportamento e os dados do domínio da aplicação, responde a solicitações de informações sobre seu estado (geralmente vindas da *view*) e a instruções para alterar esse estado (normalmente vindas do *controller*).” [BURBECK 1992]

Nessa parte inicial do projeto, a camada *View* ainda não foi implementada, visto que a parte gráfica, JavaFX, será abordada apenas na terceira e última fase.

2.3. Testes de Unidade

Testes de unidade são testes automatizados que verificam o comportamento de unidades individuais do código, como métodos e classes, de forma isolada. Neste projeto, foram utilizados com o framework JUnit 5, permitindo identificar bugs como divisão por zero, atribuições incorretas de atributos e comportamentos inesperados nas ações do jogador.

3. Metodologia

O desenvolvimento do BixoQuest partiu da análise do roteiro apresentado para a turma no início do problema, seguindo as funcionalidades apresentadas durante as discussões e modelagem. Nessa primeira fase, foram implementados os requisitos essenciais do jogo: **A criação do Jogador** e seus atributos e métodos, a **modelagem** das classes **Eventos**, **Locais e Personagens**, e o critério de **finalização** do jogo por meio da **formatura**. Todos esses requisitos foram implementados, testados no JUnit e documentados no diagrama de classes.

3.1. Uso de Inteligência Artificial

Durante o desenvolvimento deste projeto, foi utilizada a ferramenta de inteligência artificial generativa Claude (Anthropic) como suporte ao desenvolvimento. A IA foi utilizada para: identificação e correção de bugs no código, discussão de decisões de design e arquitetura, revisão de testes de unidade e auxílio na escrita da documentação. Todo o código foi compreendido, revisado e validado pelo autor antes de ser incorporado ao projeto.

3.2. Modelagem e Criação das classes

A modelagem e criação das classes partiu diretamente dos requisitos abordados no papel do problema, que já indicava os principais componentes que deveriam estar presentes:

- Jogador
- Personagem

- Locais
- Eventos

O maior desafio, então, era como organizar essas classes de forma que tivesse um baixo acoplamento e alta coesão, aproveitando os princípios de Java e POO estudados.

A classe **Personagem** foi definida como abstrata, servindo de base para *Jogador* e *NPC*, visto que ambos compartilham atributos comuns como nome, energia e saúde, mas possuem comportamentos distintos. Essa decisão evita repetição de código e estabelece uma hierarquia clara no sistema. De forma análoga, a classe **Eventos** foi especializada em duas subclasses abstratas: *EventosAleatorios* e *EventosObrigatorios*, cada uma com suas subclasses concretas que aplicam os eventos ao Jogador, garantindo o polimorfismo. A classe **Local** seguiu o mesmo princípio, ramificando-se em classes mais especializadas de cada local do jogo. Essa relação na linguagem de programação Java é expressa com o uso de *extends* e métodos abstratos no código.

```
public abstract class Eventos { 2 usages 14 inheritors phaywanon
    protected String descricao; 2 usages

    public Eventos(String descricao) { 2 usages phaywanon
        this.descricao = descricao;
    }

    public abstract void aplicarEvento(Jogador jogador); 12 implementations phaywanon
}
```

Figure 1. Classe Eventos

```
package jogo.model;

public abstract class EventosAleatorios extends Eventos { phaywanon
    public EventosAleatorios(String nome) { super(nome); }
}
```

Figure 2. Classe Eventos Aleatórios

```
package jogo.model;

public class EventoALGanhouDinheiro extends EventosAleatorios { 1 usage phaywanon
    public EventoALGanhouDinheiro() { super(nome: "Ganhou dinheiro"); }

    @Override phaywanon
    public void aplicarEvento(Jogador jogador) {
        System.out.println("Você achou 20 reais no chão!");
        jogador.setDinheiro(jogador.getDinheiro() + 20);
    }
}
```

Figure 3. Classe Evento Ganhou Dinheiro

3.3. Decisões técnicas

Algumas das decisões técnicas impactam diretamente o projeto e valem a pena ser destacadas neste documento. A classe **Mapa** foi criada para centralizar a instanciação de todos os locais, evitando que outras classes ficassem criando novos objetos com *new* repetidamente, economizando, portanto, memória e tempo de execução, tornando o programa mais eficiente. Caso contrário, o princípio do baixo acoplamento seria ferido.

```

package jogo.model;

public class Mapa {
    private LocalCantina cantina;
    private LocalCasa casa;
    private LocalColegiado colegiado;
    private LocalLaboratorio laboratorio;
    private LocalSalaDeAula salaDeAula;
    private LocalPontoDeOnibus pontoDeOnibus;
    private LocalDA da;

    public Mapa() {
        this.cantina = new LocalCantina();
        this.casa = new LocalCasa();
        this.colegiado = new LocalColegiado();
        this.laboratorio = new LocalLaboratorio();
        this.salaDeAula = new LocalSalaDeAula();
        this.pontoDeOnibus = new LocalPontoDeOnibus();
        this.da = new LocalDA();
    }

    public LocalCantina getCantina() { return cantina; }
    public LocalCasa getCasa() { return casa; }
    public LocalColegiado getColegiado() { return colegiado; }
    public LocalLaboratorio getLaboratorio() { return laboratorio; }
    public LocalSalaDeAula getSalaDeAula() { return salaDeAula; }
    public LocalPontoDeOnibus getPontoDeOnibus() { return pontoDeOnibus; }
    public LocalDA getDa() { return da; }
}

```

Figure 4. Classe Mapa

A passagem de tempo foi modelada de forma que o dia só avança quando o jogador vai para casa, tornando a simulação mais realista e dando ao jogador controle sobre o ritmo do jogo. Os eventos obrigatórios são verificados antes do incremento do dia, decisão necessária para garantir que as provas ocorram no dia correto do semestre.

O atributo *foiParaSalaHoje* foi criado para registrar se o jogador frequentou a sala de aula durante o dia, sendo utilizado pelo *EventoOBProva* para controlar se o jogador realizou ou não a prova. Esse atributo foi mantido em **Jogador**, e não no controller, pois trata-se de um estado do personagem.

Os atributos *notaAcumulada* e *provasFeitas* foram implementados para permitir o cálculo da média ao final do semestre, determinando a aprovação ou reprovação do jogador. Por fim, o atributo *formado* é definido como *true* quando o progresso do jogador atinge 100%, encerrando o jogo.

```

package jogo.model;

public class Jogador extends Personagem {
    // Atributos
    private double nivelDeConhecimento;
    private int conhecimentoSemestre;
    private int motivacao = 100;
    private double dinheiro;
    private double desempenhoAcademico = 0.0;
    private double progresso = 0.0;
    private Local local;
    private double notaAcumulada = 0.0;
    private int provasFeitas = 0;
    private boolean formado = false;
    private boolean foiParaSalaHoje = false;
}

```

Figure 5. Atributos do Jogador

3.4. Testes de Unidade

Os testes foram desenvolvidos com JUnit 5, organizados nos pacotes *teste.model* e *teste.controller*, cobrindo as principais classes do model. Entre os bugs identificados pelos testes estão: a atribuição incorreta de *motivacao* no setter de *desempenhoAcademico*, a ausência de *return* no *EventoOBFimDeSemestre* que causava divisão por zero quando

o jogador não havia realizado provas, e o *setFormado(true)* fora do bloco condicional no *EventoOBFormatura*, que formava o jogador independentemente do progresso. Esses erros foram identificados e corrigidos a partir dos testes, demonstrando a importância da prática de testes de unidade no desenvolvimento de software.

4. Conclusão

O desenvolvimento da primeira fase do BixoQuest permitiu consolidar conhecimentos apreendidos na disciplina teórica de Algoritmos e Programação II, aplicando conceitos relacionados a POO, Java, padrão MVC na prática e o uso da UML, por meio da construção de um sistema funcional e jogável. Os objetivos definidos para esta fase foram cumpridos, isso é: Diagramação das classes do jogo, implementação do *Model* e testes.

Como ponto positivo, destaca-se o amplo uso de hierarquia, polimorfismo e abstração, que são essenciais para o escopo daquele que busca ser um programador mais eficiente, além de favorecer a extensibilidade do sistema e facilitar a adição de novos eventos e locais sem impacto nas classes já existentes. A prática de testes de unidade com JUnit mostrou-se essencial, permitindo identificar bugs que não seriam facilmente percebidos apenas pela execução manual do jogo. Fora isso, o planejamento prévio com a diagramação tornou a experiência desse PBL muito mais confortável na aplicação, visto que a maioria das barreiras encontradas durante o problema eram voltadas à sintaxe do Java, e não a modelagem.

Como limitações desta fase, o atributo *nivelDeConhecimento* acumula entre semestres sem ser resetado, tornando o jogo progressivamente mais fácil. Além disso, os limites dos atributos do jogador (*energia* e *motivacao* até 110, *saude* até 120) carecem de justificativa formal de balanceamento. A camada *View* ainda contém instruções de saída no *controller*, o que será corrigido na Fase 3 com a implementação da interface gráfica em JavaFX. Desse modo, para a execução do jogo nessa fase inicial, o *Controller* acabou misturando funções que poderiam se enquadrar como *View*, algo que foi feito com o puro e único objetivo do jogo ser executável no terminal, adiantando algumas fases.

Como trabalhos futuros, para as fases seguintes, estão previstos: a implementação da persistência de dados permitindo salvar e carregar partidas, o suporte a múltiplos jogos simultâneos, a interface gráfica com JavaFX e a aplicação de ao menos três padrões de projeto. Além disso, pretende-se revisar o balanceamento dos atributos e corrigir o acúmulo de conhecimento entre semestres, tornando o jogo mais desafiador e equilibrado ao longo de toda a graduação simulada.

5. Referências

- BOOCH, G., Rumbaugh, J., and Jacobson, I. (2005). *UML: Guia do usuário*. Campus.
- BURBECK, S. (1992). Applications programming in smalltalk-80(tm):how to use model-view-controller (mvc). *Smalltalk-80*.

6. Anexos



Figure 6. Diagrama de Classes BixoQuest